

DETAILED PROJECT REPORT

Design and Implementation of a Permissioned Blockchain-Based Medical Record Integrity System

Author: Debarun Biswas

Project Repository: Final_Project

Report Type: Updated thesis-style technical report

Date: May 2026

ABSTRACT

This project presents the design and implementation of a custom permissioned blockchain system for securing medical record integrity. The system is built around a hybrid storage model: sensitive medical files are stored off-chain, while cryptographic hashes, metadata, validator identity, timestamps, and block linkage information are stored on a blockchain ledger. The purpose is not to replace every part of a hospital Electronic Health Record system, but to provide a tamper-evident trust layer that can prove whether a medical record has remained unchanged after it was registered.

The project has evolved beyond the earlier report. The original thesis focused mainly on a C-based blockchain core with Proof of Authority validation. The current implementation now includes three major layers: a C blockchain core, a Node.js/Express API bridge, and a Vite React/TypeScript dashboard. The C core manages blocks, transactions, hashing, local persistence, peer-to-peer networking, synchronization, duplicate record detection, and a demonstration signature mechanism. The Express API exposes blockchain data, network health, logs, and record upload endpoints to the web layer. The React frontend provides role-based pages for administrators, doctors, and patients, including dashboard statistics, block exploration, network monitoring, record viewing, record upload, activity logs, and integrity verification workflows.

The system follows a permissioned architecture suited to a healthcare consortium where participants are known entities such as hospitals, laboratories, clinics, insurance partners, or regulator-approved validator nodes. It uses a Proof of Authority style model instead of Proof of Work, because medical systems require low latency, predictable behavior, and identity-based accountability rather than energy-intensive mining. The implemented network flow also includes a three-phase-commit-inspired consensus sequence using CAN_COMMIT, VOTE_YES, PRE_COMMIT, ACK_PRE, and DO_COMMIT messages. This gives the project a stronger distributed-systems character than the earlier version, although the current implementation remains a prototype and not a production-grade Byzantine fault tolerant system.

The unique contribution of the project lies in combining a lightweight C ledger engine with a modern hospital-facing web application. Many blockchain demonstrations are either only theoretical, only smart-contract based, or only command-line tools. This project attempts to cover the full vertical path: low-level block storage, peer networking, backend service bridge,

and frontend operational dashboard. It also uses off-chain storage and on-chain hashes, which is important for healthcare privacy because raw patient data should not be replicated across every node in a blockchain network.

At the same time, the current system has limitations. The signature layer is presently a mock demonstration signature rather than real RSA/ECDSA verification. The backend authentication middleware supports both mock user IDs (via the X-Mock-User-ID header) and Supabase bearer tokens, allowing the existing frontend login workflow to operate seamlessly with protected upload routes. The upload flow is now fully integrated end to end: the API receives the file, copies it into the demo node's off-chain storage, calls the C viewer binary ADD command with the correct arguments, and returns the committed block index, SHA-256 hash, and data pointer to the frontend. The verify flow is also integrated: the API passes the uploaded file to the C viewer VERIFY_FILE command, which scans the chain and returns MATCH or NOT_FOUND with full metadata. Transport security, production key management, database hardening, rate limiting, test coverage, and real healthcare compliance controls remain future work. These limitations do not invalidate the academic value of the project; rather, they define the boundary between the current prototype and a production-ready healthcare infrastructure system.

TABLE OF CONTENTS

1. Introduction
2. Project Motivation
3. Problem Statement
4. Objectives and Scope
5. What the Project Is
6. What Is Unique About the Project
7. Current Project Architecture
8. C Blockchain Core
9. Network and Consensus Layer
10. Backend API Bridge
11. Frontend Dashboard
12. Data Model and Storage Design
13. Transaction Lifecycle
14. Security Model
15. Current Good Qualities
16. Current Weaknesses and Gaps
17. Differences from the Previous Report
18. Testing and Verification Status
19. Compliance and Ethical Considerations
20. Future Improvements
21. Production Roadmap
22. Conclusion
23. Appendix A: Suggested Diagrams

- 24. Appendix B: Important Files
- 25. Comparative Analysis with Existing Approaches
- 26. Formal Model of the Ledger
- 27. Design Tradeoff Analysis
- 28. Evaluation Methodology for Final Thesis
- 29. Risk Register

1. INTRODUCTION

Healthcare data is one of the most sensitive classes of digital information. A medical record may contain diagnosis history, prescriptions, laboratory reports, imaging results, allergies, surgical notes, insurance references, and long-term patient identifiers. The correctness of this data directly affects patient safety. If a medical record is altered, deleted, forged, or silently replaced, the consequence may not be only financial loss or administrative confusion; it can affect clinical decisions.

Most healthcare institutions still depend on centralized Electronic Health Record systems. These systems are efficient for storage and retrieval, but they place a large amount of trust in one database, one administrative domain, or one privileged operator group. If that trusted center is compromised, the stored record and the audit trail may both become unreliable. A database administrator, malware operator, insider attacker, or stolen privileged credential can potentially change records and logs in the same system.

Blockchain technology provides a different approach to integrity. A blockchain is an append-only sequence of blocks where each block contains a cryptographic hash of the previous block. If an older block is modified, its hash changes. This breaks the link to the next block and allows tampering to be detected. For medical data, this does not mean raw records should be placed directly on-chain. Raw medical data must remain confidential and must be handled under strict legal and ethical constraints. Therefore, this project uses a hybrid architecture: store the sensitive medical file off-chain, store only the hash and metadata on-chain, and use the blockchain as an integrity anchor.

The project is a custom implementation rather than a direct use of Ethereum, Hyperledger Fabric, or another blockchain platform. This makes it academically valuable because the internal mechanisms are visible: block creation, hash computation, block verification, disk persistence, socket networking, synchronization, and frontend presentation are all implemented in the repository. The project is not just a smart contract or a database wrapper. It is a full stack prototype that shows how a medical record integrity system can be built from the systems layer upward.

2. PROJECT MOTIVATION

The motivation comes from the mismatch between the importance of medical records and the

weaknesses of centralized trust. In a hospital environment, record integrity matters for several reasons.

First, patient safety depends on accurate history. If an allergy, medication, diagnosis, or lab report is modified, a doctor may make the wrong decision. Even a small change in a stored report can have clinical consequences.

Second, hospitals and laboratories need accountability. When a doctor, lab technician, or institution creates a record, there should be a way to prove when it was created and whether the stored file still matches the original.

Third, medical data often crosses institutional boundaries. A patient may move between hospitals, labs, clinics, insurers, and pharmacies. Each organization may store its own version of the record. Without a shared integrity anchor, disagreement between versions becomes difficult to resolve.

Fourth, healthcare systems are common ransomware targets. Attackers often try to encrypt or modify hospital databases. A tamper-evident ledger cannot prevent every attack, but it can help detect unauthorized modification and support recovery validation.

Fifth, public blockchains are not ideal for medical records. They expose too much data, operate under anonymous participation, and often have high transaction costs or unpredictable latency. A permissioned blockchain, where validators are known and approved, is more suitable for healthcare.

The project therefore explores a middle path: a private, permissioned, lightweight blockchain ledger used specifically for medical record integrity.

3. PROBLEM STATEMENT

The central problem addressed by this project is:

How can a healthcare system maintain a tamper-evident, auditable, and distributed record of medical file integrity without exposing sensitive medical data directly on a blockchain?

This problem can be divided into several sub-problems:

1. How can a medical file be represented on-chain without storing the file itself?
2. How can block history be protected against undetected modification?
3. How can validator identity be represented in a permissioned network?
4. How can duplicate records or replayed file hashes be detected?
5. How can multiple local nodes synchronize blockchain state?
6. How can the blockchain core be exposed to a web dashboard?
7. How can administrators, doctors, and patients view or interact with the system?

8. What limitations must be resolved before production use?

The project currently answers these questions at prototype level. It implements the core ledger and dashboard concepts, while leaving several security-hardening and production-integration tasks for future work.

4. OBJECTIVES AND SCOPE

The main objectives of the project are:

1. Build a custom blockchain core for medical record integrity.
2. Store medical file hashes on-chain while keeping files off-chain.
3. Provide block hash chaining using SHA-256.
4. Provide validator identity fields and demonstration block signatures.
5. Support local disk persistence of the chain.
6. Support peer-to-peer node communication over TCP.
7. Support synchronization of missing blocks between nodes.
8. Implement duplicate transaction detection through indexed hashes.
9. Build an Express API bridge between the C core and the frontend.
10. Build a React dashboard for users to inspect chain, records, logs, and node health.
11. Document strengths, weaknesses, and future improvements.

The scope is intentionally focused on integrity rather than full hospital data management. The system does not attempt to replace an EHR. It does not implement all clinical workflows. It does not store raw medical content on-chain. It is best understood as a tamper-evident integrity and audit layer that could sit beside existing hospital systems.

5. WHAT THE PROJECT IS

The project is a full stack prototype of a permissioned medical blockchain system. It has three primary components.

5.1 C Blockchain Core

The C core is located mainly under:

backend/core/src

It defines the block and transaction structures, computes SHA-256 hashes, stores blocks in a local binary ledger file, provides a viewer tool, validates the chain, and runs networked node instances. It also stores key files and off-chain encrypted record files under the core directory or demo instance folders.

5.2 Express API

The API is located under:

backend/api

It uses Node.js and Express. Its role is to expose blockchain information to the frontend. It runs the C viewer binary through child processes, parses the text output, and returns JSON. It also scans node ports to report network health and reads log files from demo node instances.

5.3 React Frontend

The frontend is located under:

frontend

It is a Vite React TypeScript application. It provides pages for:

- Login
- Dashboard
- Block Explorer
- Network Health
- System Logs
- Upload Record
- My Records / All Records
- Verify Record
- Patient Search
- Activity Logs
- Settings

The frontend gives the project an operational face. Instead of only being a command-line blockchain, the project can be demonstrated as a hospital-style web dashboard.

6. WHAT IS UNIQUE ABOUT THE PROJECT

The project is unique in several important ways.

6.1 It Is Purpose-Built for Medical Record Integrity

Many blockchain projects are general-purpose. This project is specifically designed around a healthcare use case. The transaction structure includes patient identifiers, doctor identifiers, a data hash, a data pointer, and a timestamp. This makes the chain behave like an integrity ledger for medical documents.

6.2 It Uses Hybrid Storage

The project does not store raw medical records on-chain. This is a strong design decision. Raw

medical data should not be permanently replicated across all nodes. Instead, the chain stores a hash of the encrypted file and a pointer to the off-chain location. This creates tamper evidence without unnecessary exposure.

6.3 It Combines Low-Level C with a Modern Web Stack

The blockchain engine is implemented in C, while the dashboard is implemented in React and the API in Node.js. This combination is unusual and educational. It demonstrates how a low-level ledger engine can be wrapped by a web API and displayed through a modern UI.

6.4 It Avoids Proof of Work

The project uses a permissioned Proof of Authority style approach. This is better suited for hospitals than Proof of Work because validators are known entities. There is no need for mining, gas fees, or anonymous competition.

6.5 It Includes a 3PC-Inspired Network Flow

The current node protocol includes messages such as CAN_COMMIT, VOTE_YES, PRE_COMMIT, ACK_PRE, and DO_COMMIT. This resembles a three-phase commit style flow. It is not a full Byzantine consensus protocol, but it shows distributed commit thinking and gives the project more depth than a single-node blockchain.

6.6 It Provides a Demonstrable Dashboard

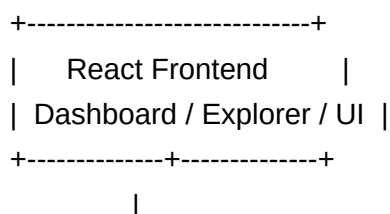
The dashboard allows the system to be shown to non-technical users. Administrators can view chain height, blocks, nodes, and logs. Doctors can upload records and inspect activity. Patients can see their records and verify files. Even if some flows are still mock or incomplete, the interface communicates the intended product clearly.

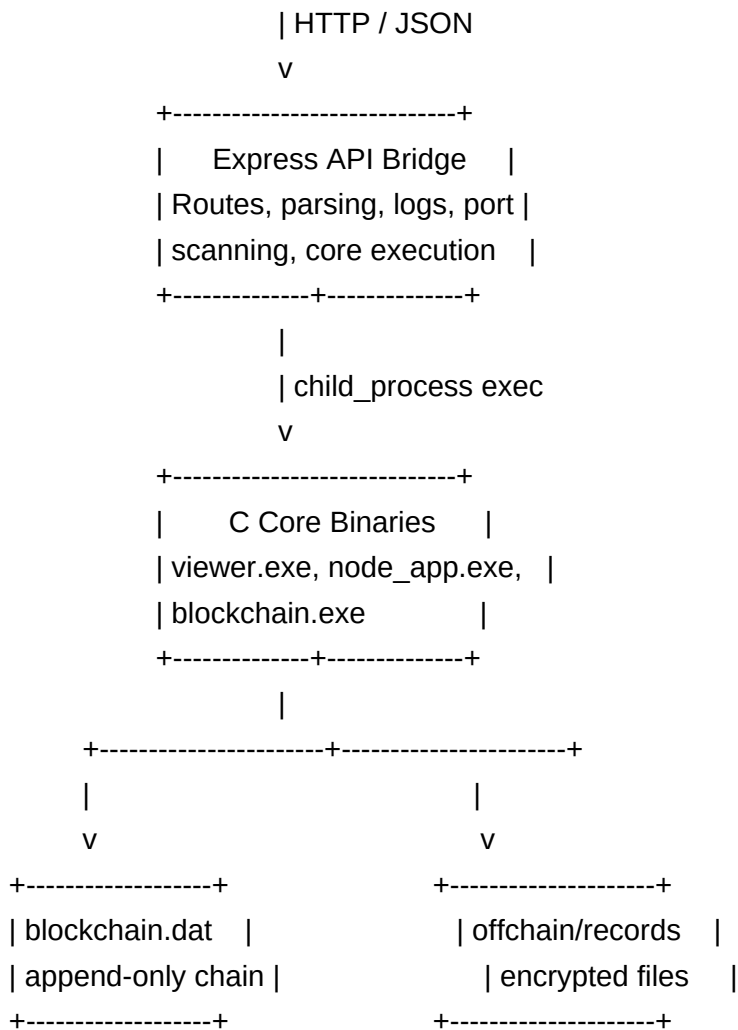
6.7 It Exposes Real Engineering Tradeoffs

The project is not hidden behind a mature enterprise framework. Its design decisions are visible. This makes it suitable for academic discussion because good and bad parts can both be studied.

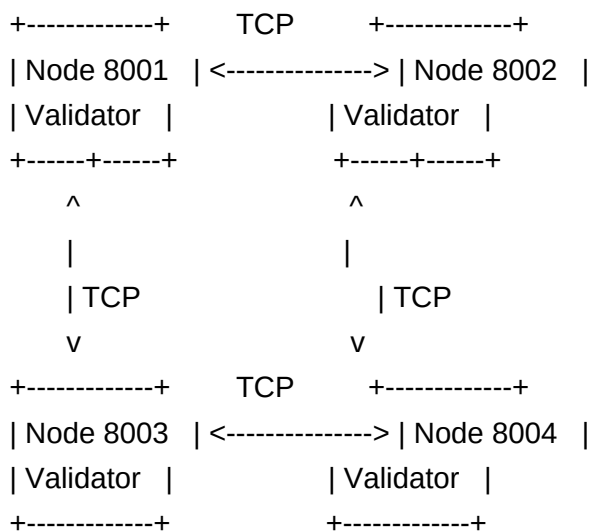
7. CURRENT PROJECT ARCHITECTURE

The current architecture can be represented as:





The distributed demo network can be represented as:



Each node has its own:

- node_app.exe binary
- keys directory
- data/blockchain.dat

- data/network.log
- offchain/records directory

The API currently reads the live demo node path first, especially demo_instances/node1, and falls back to backend/core/data in some situations.

8. C BLOCKCHAIN CORE

The C core is the foundation of the system.

8.1 Main Data Structures

The Transaction structure stores:

- patient_id
- doctor_id
- data_hash
- data_pointer
- timestamp

The Block structure stores:

- index
- timestamp
- previous_hash
- block_hash
- merkle_root
- validator_signature
- validator_port
- transactions
- transaction_count

These structures are fixed-size and simple. This is useful for deterministic file storage and easier serialization, although it also creates limits, such as fixed maximum transaction count and fixed string lengths.

8.2 Hashing

The project includes a standalone SHA-256 implementation in C. It is used for:

- hashing text data
- hashing file contents
- computing transaction hashes
- computing Merkle roots

- computing block hashes

The block hash is computed from:

- block index
- timestamp
- previous hash
- Merkle root

The Merkle root is computed from transaction fields. This means that if a transaction changes, the Merkle root changes, and therefore the block hash changes.

8.3 Genesis Block

The chain begins with a genesis block. The current implementation uses a fixed timestamp and a universal genesis validator port of 8001. The genesis transaction uses `patient_id = GENESIS` and `doctor_id = NETWORK`. This creates a predictable starting point for all nodes.

8.4 Persistence

Blocks are appended to a binary file:

`data/blockchain.dat`

The `add_block` function writes the block, flushes the file, and uses an explicit disk sync:

- `_commit` on Windows
- `fsync` on Unix-like systems

This is good because it reduces the risk of data remaining only in memory after a write.

8.5 Verification

The core verifies:

- block hash correctness
- signature prefix validity in the current demo signature implementation
- previous-hash linkage
- duplicate transaction hashes inside a block
- duplicate transaction hashes against the existing chain

Full chain verification reads blocks sequentially and checks that each block points to the previous stored hash.

8.6 Duplicate Detection

The project includes a transaction hash index implemented as a hash map with chained linked lists. This improves duplicate lookup compared to scanning the whole chain every time. Recent corrections also clear the index when the blockchain file path changes, which matters because demo nodes may use different ledger files.

8.7 Viewer Tool

The viewer binary provides commands such as:

- HEIGHT
- LAST
- ALL
- VERIFY
- PRINT <index>

The Express API depends heavily on this viewer output. It parses the text format and converts it to JSON for the frontend.

8.8 Key Generator

The current key generator creates demonstration key files. Earlier reports described OpenSSL RSA generation. The current source has moved toward a zero-dependency demonstration mode. This means the system currently demonstrates the workflow of key files and validator identity, but it does not provide production cryptographic signing.

9. NETWORK AND CONSENSUS LAYER

The network layer is implemented in C using TCP sockets. It supports both Windows and Unix-like environments through conditional compilation.

9.1 Peer Connections

Each node starts a server socket and accepts incoming peer connections. It can also connect to peer ports passed through the command line. Nodes identify themselves using MY_PORT messages.

9.2 Synchronization

The synchronization flow includes:

1. A node requests or receives CHAIN_HEIGHT.
2. If a peer has a higher height, the local node requests missing blocks using GET_BLOCK:<index>.
3. The peer sends SYNC_BLOCK messages.
4. The receiving node deserializes, verifies, and appends missing blocks.

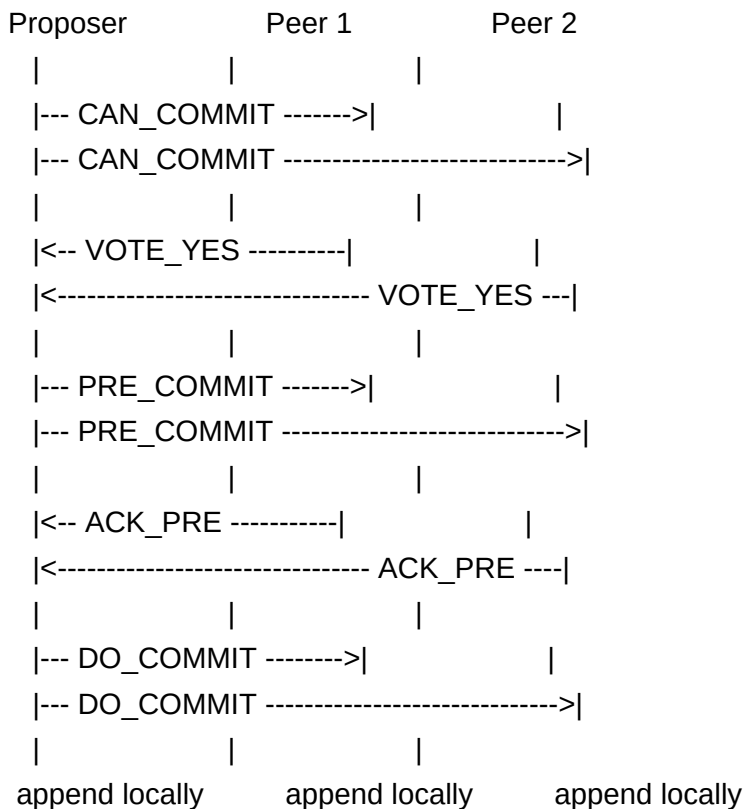
This design allows a lagging node to catch up from peers.

9.3 Three-Phase-Commit-Inspired Flow

The most important current network flow is:

1. Proposer creates a block.
2. Proposer broadcasts CAN_COMMIT:<serialized block>.
3. Peers verify the block.
4. Peers send VOTE_YES:<port>.
5. Proposer waits for enough unique votes.
6. Proposer sends PRE_COMMIT.
7. Peers reply ACK_PRE:<port>.
8. Proposer waits for enough ACK messages.
9. Proposer appends the block and broadcasts DO_COMMIT.
10. Peers append their pending block when DO_COMMIT arrives.

ASCII sequence:



9.4 Current Consensus Strength

This is a useful prototype consensus workflow. It shows that peers do not blindly accept any block. They validate and vote. It also avoids accepting duplicate votes through a vote mask.

9.5 Current Consensus Limitations

The current consensus is not production Byzantine fault tolerant. It assumes mostly honest nodes. It does not solve all cases of network partition, equivocation, malicious proposer behavior, view change, validator slashing, or formal leader election. The majority threshold is currently coded around a small demo network. A production system would need a configurable validator set and formal quorum rules.

10. BACKEND API BRIDGE

The backend API is a Node.js/Express service.

10.1 Purpose

The API exists because the frontend cannot directly read binary blockchain files or execute C programs safely. The API acts as a bridge between browser and core.

10.2 Main Routes

The blockchain routes include:

- GET /api/height
- GET /api/blocks
- GET /api/block/:index
- GET /api/verify

The network routes include:

- GET /api/network/health
- GET /api/network/logs

The record route includes:

- POST /api/records/upload

10.3 Core Bridge

The API runs the C viewer executable through `child_process.exec`. It first attempts to run commands from a live demo node directory, especially `demo_instances/node1`. It then parses viewer output.

This approach is simple and effective for a demo. It also keeps the C core separate from the JavaScript server. However, it is not ideal for production because child process execution is slower and riskier than a structured native interface, RPC service, or database-backed bridge.

10.4 Parser

The parser reads lines such as:

```
INDEX|...
TIMESTAMP|...
PREV_HASH|...
BLOCK_HASH|...
MERKLE|...
VALIDATOR|...
SIGNATURE|...
TX_COUNT|...
TX|...
---END_BLOCK---
```

It converts them into JSON blocks with nested transactions.

10.5 Network Health

The network health controller scans ports 8001 through 8010 on localhost. Any port accepting a TCP connection is treated as an online node. This is practical for the local demo and dashboard. For production, node discovery should come from a signed registry or service discovery layer rather than port scanning.

10.6 Logs

The API reads:

- backend API server logs
- demo_instances/node*/data/network.log

It aggregates these logs for the frontend System Logs page.

10.7 Authentication

The backend includes Supabase-based authentication middleware for protected routes. It expects a Bearer token and reads user role information from a profiles table. The frontend, however, currently uses local mock users and stores session data in localStorage. This creates an integration gap. The frontend and backend authentication models need to be unified.

11. FRONTEND DASHBOARD

The frontend is a React TypeScript dashboard.

11.1 Main User Roles

The application distinguishes:

- patient
- doctor
- admin

Each role sees different sidebar links and allowed pages.

11.2 Dashboard

The dashboard displays:

- chain height
- network status
- active node count
- recent blocks

This gives a quick operational overview.

11.3 Block Explorer

The block explorer displays blocks, hashes, previous hashes, Merkle roots, validator ports, transaction count, and metadata. It also includes a "Verify Full Chain" action that calls the backend verification route.

11.4 Network Health

The network page displays online validator nodes, ports, type, status, and latency. It is useful for showing that the blockchain is not only a data structure but a distributed network.

11.5 System Logs

The system logs page merges API logs and node logs. This supports audit and debugging.

11.6 Upload Record

The upload page lets a doctor or admin enter:

- patient ID
- doctor ID
- encryption password
- medical file

It sends the file through FormData to the /api/records/upload endpoint. The backend copies the file into the demo node's offchain/records directory and calls the C viewer ADD command with the file path, data pointer, patient_id, doctor_id, and validator port. On success, the frontend displays the committed block index, SHA-256 hash, data pointer, patient ID, and doctor ID returned by the core. Duplicate file submissions are detected and rejected with a 409

response.

11.7 My Records

The records page reads blockchain blocks and extracts transactions. If the user is a patient, it filters records by `patient_id`. Admins can see all records.

11.8 Verify Record

The verify page uploads the file to `/api/records/verify`. The backend passes the file to the C viewer `VERIFY_FILE` command, which recomputes the SHA-256 hash of the uploaded file and scans every block and transaction in the chain. If a matching `data_hash` is found, the response includes the block index, transaction index, hash, data pointer, patient ID, and doctor ID, all of which are displayed to the user. If no match is found, the computed hash is shown alongside a clear failure message.

11.9 Activity Logs

The activity logs page converts block and transaction history into audit events such as `BLOCK_SIGNED` and `RECORD_UPLOAD`.

11.10 Settings

The settings page is mostly a static UI placeholder for profile, security, notifications, storage configuration, and public key display.

12. DATA MODEL AND STORAGE DESIGN

12.1 On-Chain Data

The chain stores:

- block index
- block timestamp
- previous block hash
- block hash
- Merkle root
- validator signature
- validator port
- transaction array

Each transaction stores:

- `patient_id`
- `doctor_id`

- data_hash
- data_pointer
- timestamp

12.2 Off-Chain Data

Encrypted medical files are stored in offchain/records folders. The data_pointer field points to these files. The blockchain does not need the file contents to preserve integrity; it only needs the file hash.

12.3 Why Hybrid Storage Is Important

If raw medical files were stored directly on-chain:

- every node would store patient data
- deletion would become extremely difficult
- privacy exposure would increase
- chain size would grow rapidly
- compliance concerns would become serious

With hybrid storage:

- the chain remains lightweight
- private files remain outside the ledger
- file integrity can still be verified
- deletion or access revocation is more realistic

12.4 Merkle Root

The Merkle root summarizes all transactions in a block. If a transaction changes, the Merkle root changes. Since the block hash includes the Merkle root, transaction tampering changes the block hash and breaks verification.

13. TRANSACTION LIFECYCLE

The ideal transaction lifecycle is:

1. Doctor selects a medical file.
2. System encrypts or receives an encrypted file.
3. System computes SHA-256 hash of the encrypted file.
4. System creates a transaction containing patient_id, doctor_id, data_hash, pointer, and timestamp.
5. Node checks whether data_hash already exists.
6. Node creates a block.

7. Node computes Merkle root.
8. Node computes block hash.
9. Validator signs the block hash.
10. Node broadcasts CAN_COMMIT.
11. Peers verify block.
12. Peers vote yes if valid.
13. Proposer sends PRE_COMMIT.
14. Peers acknowledge.
15. Proposer and peers commit the block.
16. Frontend dashboard displays the new block and record.
17. Later, a verifier recomputes the file hash and compares it to the stored hash.

Current implementation status:

- Steps 3 through 15 exist in the C core/node flow for command-line record addition.
- Dashboard display exists.
- Upload from frontend to backend is now fully integrated: the API controller receives the file, copies it to offchain/records, calls the C viewer ADD command, and returns the committed block data to the frontend UI.
- Frontend verification is now integrated: the file is uploaded to the backend, the C viewer VERIFY_FILE command recomputes the hash and searches the chain, and the result (MATCH or NOT_FOUND) is displayed with full block/transaction details.

14. SECURITY MODEL

14.1 Security Goals

The project targets:

- integrity
- tamper evidence
- validator accountability
- duplicate detection
- auditability
- privacy by not storing raw medical data on-chain

14.2 Tamper Evidence

Tamper evidence is provided by:

- SHA-256 transaction hashes
- Merkle roots
- previous_hash linkage
- full chain verification

Changing one stored block changes its hash. The next block still points to the old hash, so verification fails.

14.3 Duplicate Detection

The system indexes transaction hashes. If the same file hash already exists, the node can reject the record. This helps prevent replay of identical medical documents.

14.4 Validator Identity

Each block stores `validator_port` and `validator_signature`. In a production system, validator identity should be tied to a real public key, certificate, and institution identity. In the current demo, signature verification is simplified.

14.5 Current Signature Limitation

The current signature implementation is not cryptographically secure. `sign_data` creates a string beginning with `SIG_V1_` and `verify_signature` checks that prefix. This is acceptable only for demonstrating the block signing workflow. It is not sufficient for real security.

A production version should use:

- Ed25519, ECDSA, or RSA-PSS signatures
- real public/private key verification
- certificate management
- key rotation
- revocation list
- hardware-backed private keys for validators

14.6 Network Security

Current node communication is plain TCP. It does not use TLS. This means traffic can be observed or modified by an attacker on the network. The block verification layer can detect some invalid data, but confidentiality and authenticity of transport are not guaranteed.

Production should use:

- TLS or mutual TLS
- node certificates
- signed peer registry
- replay protection
- message authentication
- rate limiting

14.7 Backend Security

The API uses `child_process` execution to call C binaries. This is acceptable for a controlled demo but requires hardening. Commands must be strictly controlled to avoid injection risk. In the current code, commands are mostly predefined, but future features should avoid passing raw user input into shell commands.

14.8 Frontend Security

The frontend currently uses mock authentication. This is suitable for demonstration and local testing but not for real deployment. Production should integrate actual Supabase or another authentication provider consistently across frontend and backend.

15. CURRENT GOOD QUALITIES

15.1 Clear Separation of Concerns

The project separates:

- blockchain core
- API bridge
- frontend dashboard
- documentation
- scripts
- demo instances

This makes the codebase easier to understand.

15.2 Strong Educational Value

The project exposes many important concepts:

- hashing
- block chaining
- Merkle roots
- block verification
- peer networking
- synchronization
- role-based UI
- API bridging
- off-chain storage
- audit logs

15.3 Practical Healthcare-Oriented Design

The hybrid model is correct for healthcare. Storing hashes on-chain and files off-chain is much

better than storing raw records on-chain.

15.4 Full Stack Demonstrability

The project can be shown visually through the dashboard. This makes it stronger than a command-line-only thesis project.

15.5 Low Dependency Core

The current C core avoids heavy external cryptographic dependencies in demo mode. This makes compilation easier on Windows. It also makes the internal flow easier to inspect.

15.6 Explicit Disk Sync

Using flush and disk sync after appending a block is a good reliability practice.

15.7 Duplicate Record Handling

The transaction hash index is a meaningful improvement. It turns duplicate detection from a purely linear lookup into indexed lookup.

15.8 Multi-Node Demo Structure

The demo_instances folder supports multiple local nodes, each with its own data, keys, logs, and records. This is useful for demonstration.

15.9 Dashboard Breadth

The frontend covers many expected operational views:

- dashboard
- explorer
- network
- logs
- upload
- records
- verification
- settings

This gives the project product depth.

16. CURRENT WEAKNESSES AND GAPS

16.1 Mock Signatures

The largest security weakness is that the signature system is not real cryptography. It demonstrates the shape of signing but does not protect against forgery.

16.2 Auth Model Is Demo-Level

The backend authentication middleware now accepts both the X-Mock-User-ID header (used by the frontend mock login) and Supabase bearer tokens. This allows the existing frontend login workflow to work with protected routes without requiring a full Supabase deployment for the demo. A production system must unify on a single authentication provider with proper token validation, session management, and role enforcement backed by a real identity store.

16.3 Upload Flow Is Integrated at Demo Level

The upload controller now copies the uploaded file into the demo node's offchain/records directory and calls the C viewer ADD command with the correct arguments. The C core hashes the file, rejects duplicates, creates a block, signs it with the demo validator key, and appends it to blockchain.dat. The committed block index, hash, and data pointer are returned to the frontend. The remaining limitation is that the encryption step is a file copy rather than real AES encryption, because the AES utility code exists in the C core but is not yet wired into the upload pipeline.

16.4 Verification UI Is Now Integrated

The Verify Record page no longer simulates verification. It uploads the file to /api/records/verify. The backend passes the file to the C viewer VERIFY_FILE command, which recomputes the SHA-256 hash and scans the chain. A MATCH response returns the block index, transaction index, hash, data pointer, patient ID, and doctor ID. A NOT_FOUND response returns the computed hash. The frontend displays all of this information clearly.

16.5 Plain TCP Networking

Node communication is not encrypted. Production healthcare systems need TLS or mutual TLS.

16.6 Consensus Is Demo-Level

The three-phase commit inspired logic is useful but not a complete consensus protocol. It lacks formal handling for malicious validators, network partitions, leader changes, conflicting proposals, and dynamic validator sets.

16.7 Binary File Storage Has Portability Risks

The ledger writes C structs directly to disk. This is simple, but it can be sensitive to compiler, platform, architecture, struct layout, and time_t size differences. A production ledger should use a canonical binary or JSON/CBOR/Protobuf format.

16.8 Limited Automated Tests

The repository contains test sources, but the frontend and backend do not have meaningful automated test suites. backend/api npm test is a placeholder.

16.9 Logs and Demo Data Are Tracked

Demo blockchain files, node logs, and binaries appear in the repository. For production-quality repository hygiene, generated artifacts should usually be excluded or separated from source.

16.10 No Real Encryption Pipeline Yet

The architecture assumes encrypted records, and the UI asks for an encryption password, but the full encryption pipeline is not implemented end to end in the web upload flow.

16.11 No Real Patient Consent Model

A medical system should record patient consent, access grants, revocations, and audit trails. The current project focuses on integrity, not consent governance.

16.12 No Production Database Schema

The Supabase profile idea exists, but the chain and frontend are not fully integrated with a stable database schema for users, permissions, records, and audit metadata.

17. DIFFERENCES FROM THE PREVIOUS REPORT

The earlier report mainly described a permissioned C blockchain with RSA signatures and a lower-level architecture. Since then, the project has changed significantly.

17.1 Added Web Frontend

The project now includes a React TypeScript dashboard with role-based pages. This was not central in the earlier report.

17.2 Added Express API

The project now includes an API bridge that exposes the C blockchain to the frontend.

17.3 Changed Signature Reality

The previous report described RSA-2048 integration. The current implementation uses a mock signature mechanism for demo portability. The report must therefore clearly separate intended production cryptography from current demo behavior.

17.4 Added Network Health and Logs

The dashboard and backend can now display node status and logs from demo instances.

17.5 Added Duplicate Indexing

The core now includes transaction hash indexing and duplicate detection.

17.6 Added 3PC-Style Messages

The current protocol includes CAN_COMMIT, VOTE_YES, PRE_COMMIT, ACK_PRE, and DO_COMMIT.

17.7 Added Full Stack User Roles

The frontend has patient, doctor, and admin routes.

17.8 Updated Build Reality

The current C core builds through MinGW make on Windows. The README has been updated to reflect the real repository structure.

18. TESTING AND VERIFICATION STATUS

During the latest review, the following verification commands passed:

- frontend npm run lint
- frontend npm run build
- backend API JavaScript syntax checks through node --check
- backend/core mingw32-make all
- backend/core bin/viewer.exe VERIFY returns EMPTY when there is no core ledger

The frontend build produced only a Vite bundle-size warning, not a failure.

Important interpretation:

The core viewer returns EMPTY when no data/blockchain.dat exists in backend/core. Demo instances still contain their own chain files. Therefore, an empty central core chain is not the same as a corrupted demo chain.

Testing gaps:

- no API unit tests
- no frontend component tests
- no automated multi-node consensus test
- no upload integration test

- no cryptographic signature test because signatures are currently mock
- no long-running network stability test
- no fuzz testing for the serializer/parser

19. COMPLIANCE AND ETHICAL CONSIDERATIONS

19.1 Privacy

The system correctly avoids putting raw medical files on-chain. This is the most important privacy-preserving decision in the architecture.

19.2 Data Minimization

Only hashes, pointers, and limited metadata should be stored on-chain. In future versions, `patient_id` and `doctor_id` should be pseudonymized or tokenized more strongly.

19.3 Right to Erasure

Blockchain data is immutable, so deleting on-chain data is difficult. The hybrid model helps because the medical file can be deleted off-chain while the chain retains only a hash. This is often called crypto-shredding when encryption keys or encrypted files are destroyed.

19.4 Consent

The current implementation does not fully model patient consent. A production medical system should record:

- consent granted
- consent revoked
- consent scope
- who accessed a record
- why access occurred
- emergency override events

19.5 Auditability

The ledger and logs support auditability, but production audit logs need stronger guarantees. API logs should be append-only, protected from modification, and possibly anchored into the blockchain itself.

19.6 Ethical Use

A system like this should never expose raw medical data unnecessarily. It should also avoid giving a false sense of security. Blockchain protects integrity, but it does not automatically solve access control, confidentiality, consent, or clinical correctness.

20. FUTURE IMPROVEMENTS

20.1 Replace Mock Signatures with Real Cryptography

This is the highest priority.

Recommended options:

- Ed25519 for fast modern signatures
- ECDSA P-256 for standards compatibility
- RSA-PSS if RSA is required

The implementation should verify that a block was signed by the private key corresponding to the validator public key.

20.2 Use Canonical Serialization

Instead of writing C structs directly to disk, use a stable format:

- Protocol Buffers
- CBOR
- FlatBuffers
- custom fixed-width binary format

This would improve cross-platform compatibility.

20.3 Complete Upload Pipeline

The web upload should:

1. receive file
2. encrypt file or confirm encrypted file
3. store file in offchain/records
4. compute file hash
5. call a real core add command or service
6. return block/transaction result
7. update frontend state

20.4 Implement Real Verification

The Verify Record page should:

- compute SHA-256 in the browser, or
- upload the file to the backend for hashing, then
- compare the hash with chain transactions.

20.5 Unify Authentication

Choose one auth model:

- Supabase auth end to end, or
- local mock auth only for demo, or
- JWT issued by backend

The frontend and backend must agree on token format and role source.

20.6 Add TLS / Mutual TLS

Node-to-node communication should be encrypted and authenticated. In a hospital consortium, each validator should have a certificate.

20.7 Add Validator Governance

The system needs:

- validator registry
- onboarding process
- removal process
- key rotation
- revocation list
- quorum configuration

20.8 Improve Consensus

Options:

- formal Raft-style leader election for crash fault tolerance
- PBFT/Tendermint-style consensus for Byzantine settings
- configurable quorum rules
- timeout and recovery from failed proposals
- protection against double proposals

20.9 Add Database Support

The API could store user profiles, record metadata, access policies, and audit summaries in Postgres/Supabase while keeping integrity proofs on-chain.

20.10 Add Automated Tests

Recommended tests:

- hash function tests against known SHA-256 vectors

- block hash reproducibility tests
- duplicate detection tests
- chain verification tests
- serializer/deserializer tests
- API route tests
- frontend page rendering tests
- multi-node sync tests
- upload integration tests

20.11 Improve Repository Hygiene

Generated files should be controlled:

- binaries can be ignored or stored in release artifacts
- logs should not normally be committed
- demo data should be explicitly separated from source
- keys should not be real secrets in the repository

20.12 Add Monitoring

Production monitoring should include:

- node online/offline status
- block height mismatch
- sync lag
- failed block verification
- duplicate record attempts
- upload failures
- suspicious access patterns

20.13 Add Access Control and Consent

Medical data systems need strong user permissions. Future work should include:

- patient consent grants
- doctor access requests
- emergency access mode
- audit trail for every access
- role-based and attribute-based access control

20.14 Add Performance Benchmarking

The thesis should eventually include:

- multiple benchmark runs

- average latency
- standard deviation
- throughput under different node counts
- throughput under different transaction counts per block
- memory usage
- CPU usage
- network sync time
- charts and graphs

21. PRODUCTION ROADMAP

Phase 1: Stabilize Prototype

- fix upload integration
- replace simulated verification with real hash comparison
- unify authentication
- add basic automated tests
- clean generated files from repository

Phase 2: Secure Core

- implement real signatures
- implement canonical serialization
- add TLS/mTLS
- add signed validator registry
- add key rotation and revocation

Phase 3: Improve Distributed Protocol

- configurable validator set
- formal quorum
- timeout handling
- failed proposer recovery
- network partition behavior
- conflict resolution

Phase 4: Healthcare Workflow Integration

- patient consent model
- doctor/patient profile database
- hospital/lab/insurance roles
- record access auditing
- integration with existing EHR systems

Phase 5: Production Readiness

- monitoring dashboard
- deployment scripts
- containerization
- backup and restore
- disaster recovery
- security audit
- compliance documentation

22. CONCLUSION

This project is a strong academic and engineering prototype of a permissioned blockchain-based medical record integrity system. Its most important idea is the separation of medical data storage from integrity proof. Sensitive files remain off-chain, while hashes and metadata are stored in a tamper-evident ledger. This design aligns with healthcare privacy needs better than storing raw records directly on-chain.

The current repository demonstrates more than a theoretical blockchain. It includes a C ledger engine, a peer networking layer, a synchronization mechanism, a Node.js API bridge, and a React dashboard. This makes the project suitable for demonstration, explanation, and further development.

The project also honestly reveals the gap between prototype and production. It still needs real cryptographic signatures, real authentication integration, secure transport, a complete upload and verification pipeline, automated tests, formal validator governance, and stronger consensus rules. These are not minor details; they are essential for real healthcare deployment. However, the current implementation provides a meaningful foundation on which those improvements can be built.

The uniqueness of the work lies in its full-stack nature and its healthcare-specific architecture. It does not merely use blockchain as a buzzword. It uses blockchain for a specific property: tamper evidence. It does not place private data on-chain. It recognizes that permissioned identity is more suitable than public mining for hospitals. It uses a dashboard to make the system understandable to users and examiners.

In final evaluation, the project should be presented as a permissioned medical integrity ledger prototype with a working core, demonstrable dashboard, and clear future path toward production-grade security and compliance.

23. APPENDIX A: SUGGESTED DIAGRAMS

The following diagrams should be included if this report is converted into a PDF thesis.

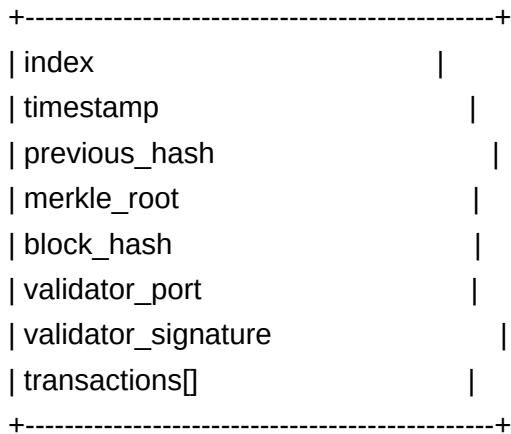
23.1 Overall Architecture Diagram

React Frontend -> Express API -> C Core Binaries -> Ledger and Off-chain Storage

23.2 Node Interaction Sequence Diagram

CAN_COMMIT -> VOTE_YES -> PRE_COMMIT -> ACK_PRE -> DO_COMMIT

23.3 Block Structure Diagram



23.4 Transaction Lifecycle Diagram

File -> Hash -> Transaction -> Block -> Consensus -> Commit -> Verify

23.5 Threat Model Diagram

Untrusted client -> API boundary -> Core boundary -> Ledger/off-chain storage

23.6 Key Flow Diagram

Validator key file -> sign block hash -> peers verify validator signature

23.7 Network Topology Diagram

Local validator nodes on ports 8001-8010 connected through TCP.

24. APPENDIX B: IMPORTANT FILES

Core blockchain:

- backend/core/src/blockchain/block.h
- backend/core/src/blockchain/block.c
- backend/core/src/blockchain/blockchain.c
- backend/core/src/crypto/hash.c

- backend/core/src/crypto/signature.c
- backend/core/src/node_app.c
- backend/core/src/viewer.c

Network:

- backend/core/src/network/node.c
- backend/core/src/network/protocol.c
- backend/core/src/network/serializer.c
- backend/core/src/network/proposal.c
- backend/core/src/network/sync.c

API:

- backend/api/src/index.js
- backend/api/src/config/core_bridge.js
- backend/api/src/controllers/blockController.js
- backend/api/src/controllers/networkController.js
- backend/api/src/controllers/recordController.js
- backend/api/src/utils/parser.js
- backend/api/src/middleware/auth.js

Frontend:

- frontend/src/App.tsx
- frontend/src/hooks/useAuth.ts
- frontend/src/services/api.ts
- frontend/src/pages/Dashboard.tsx
- frontend/src/pages/BlockExplorer.tsx
- frontend/src/pages/NetworkHealth.tsx
- frontend/src/pages/SystemLogs.tsx
- frontend/src/pages/UploadRecord.tsx
- frontend/src/pages/MyRecords.tsx
- frontend/src/pages/VerifyRecord.tsx

25. COMPARATIVE ANALYSIS WITH EXISTING APPROACHES

25.1 Centralized Hospital Database

A conventional hospital database is simple to deploy and easy to query. It gives high performance and mature tooling. However, it depends heavily on central trust. If the database administrator, storage layer, or audit log system is compromised, the same authority that stores data may also control the evidence of modification. This is the weakness the project tries to reduce.

Compared with a centralized database, this project provides stronger tamper evidence. A changed historical block should break hash verification. A record file changed outside the chain should produce a hash mismatch. The tradeoff is that blockchain introduces complexity, distributed synchronization, and more difficult data correction.

25.2 Public Blockchain

Public blockchains such as Bitcoin or Ethereum provide strong openness and censorship resistance. They are designed for environments where participants do not necessarily trust each other. However, healthcare systems usually require known participants, controlled membership, privacy, and predictable cost.

Public blockchains are not suitable for raw medical records because:

- every validator may see replicated data
- transaction fees may vary
- latency may be unpredictable
- patient privacy is difficult to guarantee
- deletion or correction is difficult

This project avoids those problems by using a permissioned model and off-chain storage.

25.3 Hyperledger Fabric

Hyperledger Fabric is a mature permissioned blockchain framework. It supports channels, chaincode, identity management, ordering services, and enterprise deployment. It is far more complete than this project. However, Fabric is also complex. It requires a larger operational stack and more infrastructure knowledge.

This project is smaller and easier to understand. It is not meant to replace Fabric in production, but it is useful academically because the internal blockchain mechanisms are visible and implemented directly.

25.4 Traditional Audit Logs

Traditional audit logs record who did what and when. They are useful, but they are usually stored in the same administrative environment as the main system. A privileged attacker may alter both data and logs. A blockchain-style append-only ledger improves this by linking every block to previous history. The audit trail becomes harder to rewrite silently.

25.5 Cloud Object Storage with Checksums

Cloud storage systems can store object hashes and version history. This is practical and scalable. However, the trust still rests largely with the cloud provider or account owner. A permissioned blockchain can create a shared institutional integrity layer where multiple

parties validate the history.

26. FORMAL MODEL OF THE LEDGER

26.1 Block Definition

Let the blockchain be an ordered sequence:

$$C = [B_0, B_1, B_2, \dots, B_n]$$

Each block B_i contains:

$$B_i = (\text{index}, \text{timestamp}, \text{previous_hash}, \text{merkle_root}, \text{block_hash}, \text{validator_id}, \text{signature}, \text{TxList})$$

For the current implementation, validator_id is represented by validator_port .

26.2 Transaction Definition

Each transaction T contains:

$$T = (\text{patient_id}, \text{doctor_id}, \text{data_hash}, \text{data_pointer}, \text{timestamp})$$

Here:

$$\text{data_hash} = \text{SHA256}(\text{file_content})$$

The file_content should ideally be encrypted medical file content. The blockchain stores the hash, not the raw medical file.

26.3 Merkle Root

For each transaction T_i , define:

$$\text{tx_hash_i} = \text{SHA256}(\text{patient_id} \parallel \text{doctor_id} \parallel \text{data_hash} \parallel \text{data_pointer} \parallel \text{timestamp})$$

The Merkle root is computed by repeatedly hashing pairs:

$$M = \text{Merkle}(\text{tx_hash_1}, \text{tx_hash_2}, \dots, \text{tx_hash_k})$$

If there is only one transaction, the Merkle root is the transaction hash. If there are multiple transactions, the root summarizes the set. If any transaction field changes, the root changes.

26.4 Block Hash

The block hash is:

$\text{block_hash} = \text{SHA256}(\text{index} \parallel \text{timestamp} \parallel \text{previous_hash} \parallel \text{merkle_root})$

This means block integrity depends on:

- block position
- block creation time
- previous block hash
- transaction summary

26.5 Chain Validity

A chain is valid if all of the following are true:

1. B_0 is a valid genesis block.
2. For every $i > 0$, $B_i.\text{previous_hash} = B_{(i-1)}.\text{block_hash}$.
3. For every block B_i , $\text{recomputed_hash}(B_i) = B_i.\text{block_hash}$.
4. For every block B_i , validator signature is valid under the validator identity rules.
5. No duplicate transaction hash violates system policy.

26.6 Integrity Claim

If SHA-256 is collision resistant and block verification is performed honestly, then modifying any committed transaction or block field should be detected because the Merkle root or block hash changes. The chain therefore provides tamper evidence, not automatic tamper prevention.

26.7 Privacy Claim

If only file hashes and pseudonymous metadata are stored on-chain, the chain does not directly reveal the medical file. However, metadata can still leak sensitive information if `patient_id` or `doctor_id` are meaningful identifiers. A production system should tokenize or pseudonymize these fields more strongly.

27. DESIGN TRADEOFF ANALYSIS

27.1 Why Proof of Authority Instead of Proof of Work?

Proof of Work is designed for open anonymous networks. Healthcare networks are not anonymous; validators are known institutions. Proof of Authority avoids mining cost, reduces latency, and improves accountability. The tradeoff is that governance becomes important. The system must carefully decide who is allowed to validate.

27.2 Why Off-Chain Storage?

Off-chain storage protects privacy and reduces blockchain size. The tradeoff is that availability of the original file depends on the off-chain storage system. If the file is deleted, the chain can still prove that a file once existed, but it cannot recover the file.

27.3 Why C for the Core?

C gives low-level control, fast execution, and transparent memory layout. It is useful for learning and systems programming. The tradeoff is that C requires careful handling of buffers, string lengths, memory allocation, and platform differences. A production version may prefer Rust or a memory-safe language for reducing memory safety risk.

27.4 Why Express as a Bridge?

Express is fast to build and easy to connect with a React frontend. The tradeoff is that the current bridge uses `child_process` execution, which is less robust than a persistent core service or native binding. A future version should expose the core through a stable RPC API.

27.5 Why React for Frontend?

React is suitable for interactive dashboards and role-based views. It makes the project more demonstrable. The tradeoff is that UI can create a false sense of completeness if backend workflows are not fully implemented. The report must therefore clearly distinguish implemented backend behavior from simulated frontend behavior.

27.6 Why Direct Binary Ledger Files?

Direct binary files are easy to append and fast to read in C. The tradeoff is portability. Different platforms may represent structures differently. A canonical serialization format would be safer.

27.7 Why Demo Signature Mode?

The current demo signature mode removes OpenSSL dependency and makes the project easier to compile. The tradeoff is security. It must be replaced with real cryptography before any serious deployment.

28. EVALUATION METHODOLOGY FOR FINAL THESIS

The final academic thesis should not only state that the system works. It should evaluate it with repeatable experiments.

28.1 Functional Tests

Recommended functional tests:

1. Create genesis block.
2. Add one valid record.
3. Add duplicate record and confirm rejection.
4. Tamper with a block and confirm verification failure.
5. Tamper with off-chain file and confirm hash mismatch.
6. Start two nodes and confirm synchronization.
7. Start four nodes and confirm consensus commit.
8. Query blocks through API and confirm frontend display.

28.2 Performance Tests

Recommended metrics:

- block creation time
- block verification time
- file hashing time by file size
- block append time
- API response time
- frontend load time
- synchronization time for N missing blocks
- throughput in blocks per second
- throughput in transactions per second

28.3 Suggested Experimental Matrix

Run tests with:

- 1 node, 10 records
- 1 node, 100 records
- 1 node, 1000 records
- 2 nodes, 100 records
- 4 nodes, 100 records
- files of 10 KB, 1 MB, and 10 MB

Each test should be repeated at least five times. The report should include average, minimum, maximum, and standard deviation.

28.4 Stress Tests

Recommended stress tests:

- repeated duplicate submissions
- rapid API polling
- node shutdown during sync
- node shutdown during proposal

- corrupted blockchain file
- missing off-chain record
- malformed network message

28.5 Security Tests

Recommended security tests:

- forged block signature
- wrong previous hash
- duplicate transaction hash
- modified transaction data
- invalid validator port
- oversized serialized message
- path traversal attempt during upload

29. RISK REGISTER

Risk 1: Mock signature accepted as real security

Impact: High

Likelihood: Medium

Mitigation: Clearly document demo status and replace with real cryptography.

Risk 2: Frontend upload appears complete but backend does not commit records

Impact: High

Likelihood: High

Mitigation: Implement a real add-record API path and integration test.

Risk 3: Plain TCP exposes node messages

Impact: Medium to High

Likelihood: Medium

Mitigation: Add TLS or mutual TLS.

Risk 4: Binary ledger portability problems

Impact: Medium

Likelihood: Medium

Mitigation: Use canonical serialization.

Risk 5: Lost off-chain files

Impact: High

Likelihood: Medium

Mitigation: Use redundant encrypted storage, backups, and file availability checks.

Risk 6: Weak patient privacy through metadata

Impact: High

Likelihood: Medium

Mitigation: Pseudonymize patient and doctor identifiers; minimize metadata.

Risk 7: Consensus failure during network partition

Impact: High

Likelihood: Medium

Mitigation: Formalize consensus, quorums, timeouts, and recovery.

Risk 8: Child process bridge injection or operational fragility

Impact: Medium

Likelihood: Medium

Mitigation: Avoid shell command composition; expose core through stable service API.

Risk 9: No automated regression tests

Impact: Medium

Likelihood: High

Mitigation: Add unit, integration, and multi-node tests.

Risk 10: Repository contains generated data and binaries

Impact: Low to Medium

Likelihood: High

Mitigation: Clean .gitignore and separate demo artifacts from source code.

END OF REPORT